[AI][Knowledge] GitFlow

1. GitFlow là gì và tại sao chúng ta cần nó?
2. Các quy tắc phân nhánh trong GitFlow
   - Nhánh vòng đời dài (Main Branches)
   - Nhánh hỗ trợ (Supporting Branches)
3. Các quy tắc Commit (Conventional Commits)
4. Ví dụ thực tế về luồng đi của code

Khi mới học lập trình, chúng ta thường quen với việc đẩy (push) thẳng mọi dòng code lên nhánh `main`. Tuy nhiên, khi bước vào môi trường doanh nghiệp hay làm việc nhóm với nhiều lập trình viên cùng lúc, thói quen này sẽ nhanh chóng dẫn đến thảm họa: conflict code liên tục, lỗi không biết do ai gây ra, và hệ thống sập không thể cứu vãn.

Đó là lúc **GitFlow** xuất hiện !!!

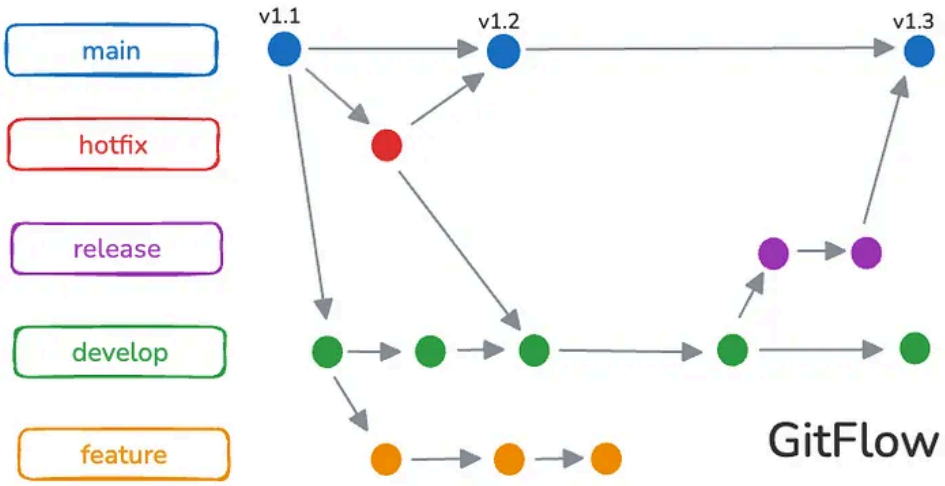
1. GitFlow là gì và tại sao chúng ta cần nó?

GitFlow là một mô hình quản lý phân nhánh (branching model) cho Git, được giới thiệu bởi Vincent Driessen vào 2010. Nó định nghĩa một bộ quy tắc nghiêm ngặt về việc tạo nhánh, tích hợp code và phát hành phiên bản.

Nhưng tại sao doanh nghiệp và các team Dev lại bắt buộc phải dùng GitFlow?

- **Tránh "dẫm chân" lên nhau:** Nhiều người có thể phát triển các tính năng khác nhau cùng lúc mà không làm ảnh hưởng đến code của người khác.
- **Luôn có một phiên bản ổn định:** Nhánh chính luôn ở trạng thái "sạch" và sẵn sàng để triển khai (deploy) cho người dùng cuối bất cứ lúc nào.
- **Dễ dàng xử lý sự cố:** Khi production xảy ra lỗi nghiêm trọng, bạn có quy trình chuẩn để sửa lỗi (hotfix) ngay lập tức mà không phải chờ các tính năng đang code dở dang.
- **Tích hợp hoàn hảo với CI/CD:** Hệ thống tự động (DevOps) có thể dựa vào tên nhánh để biết nên chạy test ở môi trường Staging hay tự động deploy lên Production.

2. Các quy tắc phân nhánh trong GitFlow



GitFlow chia các nhánh (branch) thành hai nhóm chính: **Nhánh vòng đời dài** (luôn tồn tại) và **Nhánh hỗ trợ** (tạo ra để làm việc và xóa đi khi xong).

Nhánh vòng đời dài (Main Branches)

- `main` : Đây là nhánh chính ("main" 😄). Code ở đây luôn là code đang chạy trên môi trường Production. Tuyệt đối không ai được commit trực tiếp lên nhánh này.
- `develop` : Đây là nhánh tích hợp. Mọi tính năng mới sau khi code xong đều được gộp (merge) vào đây. Khi nhánh `develop` đủ ổn định, nó sẽ được chuẩn bị "bế" lên `main` .

Nhánh hỗ trợ (Supporting Branches)

- `feature/*` (Nhánh tính năng):
  - Tách ra từ: `develop`
  - Merge trở lại: `develop`
  - Mục đích: Dùng để phát triển một tính năng cụ thể (VD: `feature/guarail-agent` , `feature/schema-linking-agent`).
- `release/*` (Nhánh phát hành):
  - Tách ra từ: `develop`
  - Merge trở lại: `main` và `develop`
  - Mục đích: Đóng băng code để chuẩn bị ra mắt phiên bản mới (VD: `release/v1.2.0`). Ở nhánh này chỉ fix các lỗi nhỏ (bug fix), cập nhật tài liệu, không thêm tính năng mới.
- `hotfix/*` (Nhánh sửa lỗi khẩn cấp):
  - Tách ra từ: `main`
  - Merge trở lại: `main` và `develop`

- Mục đích: Dùng khi Production xảy ra lỗi nghiêm trọng cần vá ngay lập tức (VD: `hotfix/fix-crash-payment`)

3. Các quy tắc Commit (Conventional Commits)

Một GitFlow tốt phải đi kèm với những commit message chuẩn mực. Chuẩn **Conventional Commits** là bắt buộc trong các dự án chuyên nghiệp.

Cấu trúc chuẩn: `<type>(<scope>): <description>`

- `feat`: Thêm tính năng mới.
- `fix`: Sửa lỗi.
- `docs`: Cập nhật tài liệu (README.md, Swagger...).
- `refactor`: Viết lại code cho tối ưu hơn nhưng không đổi logic, không thêm tính năng.
- `chore`: Cập nhật cấu hình, thư viện, dependencies...
- `test`: Viết unit test, integration test.

e.g.: refactor(guardrail-agent): improve prompt, feat(agent): add guarail agent, etc...

Quy tắc vàng:

1. Viết bằng tiếng Anh.
2. Dùng câu mệnh lệnh (VD: `add`, không dùng `added` hay `adds`).
3. Không viết hoa chữ cái đầu tiên sau dấu hai chấm.

4. Ví dụ thực tế về luồng đi của code

Xem từ đoạn: 2:08 đến 3:07 😌

Tất Tần Tật Về Git Flow Trong 3 Phút – Tự Học Siêu Tốc